APEX: A Self-Improving Harness for LLM Coding Agents Across the Software Lifecycle

Vicente Rivas Monferrer
VRAIN, Universitat Politècnica de València
València, Spain
TODO@upv.es

Juan Miguel Alberola
VRAIN, Universitat Politècnica de València
València, Spain
jalberola@dsic.upv.es

Víctor Sánchez-Anguix
VRAIN, Universitat Politècnica de València
València, Spain
visanan1@upv.es

Vicente Julián
VRAIN, Universitat Politècnica de València
València, Spain
vinglada@dsic.upv.es

ABSTRACT

LLM coding agents such as Claude Code now run multi-day, multi-project software work, yet they degrade over long horizons: they lose context, forget which tools exist, repeat errors made days ago in other projects, drift between development phases, and declare phases "done" against mock data. We present **APEX**, a harness that gives a coding agent a persistent, self-correcting cognitive substrate. APEX layers entirely on top of an existing agent's extensibility primitives—tool servers, lifecycle hooks, declarative skills, sub-agents—and adds: (i) a phase-gated software lifecycle (14 phases, from a Socratic intake to maintenance) whose transitions are guarded by deterministic-first checks; (ii) a per-project temporal context database plus a cross-project hub that re-injects past lessons at session start; and (iii) a self-improvement loop (an outcome-based grader, an advisor sub-agent, a host-introspective "doctor", and a skill forge) that proposes fixes to the harness itself. The artifact is open source, ships 620 passing tests, exposes 53 tool endpoints and 24 skills, and has been the working harness for ≈20 real software and research projects. We describe the design, walk through a session, and report what is and is not yet validated. A five-minute screencast accompanies the paper.

CCS CONCEPTS

• **Software and its engineering** → **Software development methods**; • **Computing methodologies** → *Intelligent agents.*

KEYWORDS

LLM agents, agentic software engineering, developer harness, context engineering, self-improving systems, Claude Code, Model Context Protocol

1 INTRODUCTION

General-purpose LLM coding agents—Claude Code, Cursor's agent mode, OpenAI Codex CLI, and open clones—are now used not as autocomplete but as the operator of whole software tasks: they plan, edit files, run shells, call services, and iterate over many turns [14]. As tasks lengthen from minutes to days and span several repositories, a different failure mode dominates. It is not that the model writes a bad line; it is that the *harness around the model* has no memory and no discipline. Concretely, in our own use of one such agent we repeatedly observed: (a) **context amnesia**—a decision settled on Monday re-litigated on Wednesday; (b) **forgotten capabilities**—bespoke tools and skills that exist but are never invoked; (c) **cross-project error recurrence**—the very mistake that broke project A made again in project B; (d) **phase drift**—jumping to implementation before the spec or the threat model exists; and (e) **premature "done"**—marking an integration phase complete with hard-coded mock data. Studies of multi-agent LLM systems report production failure rates of 41–87%, attributed predominantly to coordination rather than to model capability [21]; the single-agent, long-horizon case is less measured but the symptoms above are familiar to any practitioner.

Recent work names the response to this: *harness engineering* [31]—building the runtime infrastructure (memory stores, skills, protocols) that earlier systems hoped the model would reconstruct internally. "Dive into Claude Code" [14] documents the four extensibility mechanisms a modern agent exposes—tool servers (MCP), plugins, declarative skills, and lifecycle hooks—plus sub-agent delegation. The self-evolving-agents survey [9] casts the ideal system as a four-part loop (System Inputs, Agent System, Environment, Optimisers) and notes that most deployed agents are statically configured. Concurrent 2026 work now supplies each ingredient in isolation—deterministic pre-LLM phase gates [15], cross-project agent memory [16], and harness self-optimisation from past trajectories [23]—but no system we are aware of combines them: a phase-gated lifecycle with *deterministic-first* gates, a temporal context store that is both per-project and cross-project, *and* a loop that closes on judged outcomes by proposing changes to the harness itself, all at once on top of an unmodified commercial agent.

This paper presents APEX, an open-source harness that does exactly that. Our contributions are: (1) the *design* of a phase-gated agentic SDLC whose phase transitions are blocked by a hook running deterministic checks before any LLM judgement is consulted; (2) a *context substrate*—a per-project SQLite database with temporal facts, bidirectional error memory, and an FTS5 wiki, plus a cross-project hub that re-injects past lessons at session start; (3) a *self-improvement loop*—an outcome-based grader, an advisor sub-agent under a strict token/rate budget, a host-introspective "doctor", and a skill forge that mines repeated patterns into proposed new skills; and (4) the *artifact* itself: 53 tool endpoints, 24 skills, 11 hooks, 620 passing tests, MIT-licensed, archived on Zenodo, and used as the working harness for ≈20 real projects over roughly two months. A five-minute screencast demonstrates a live session.

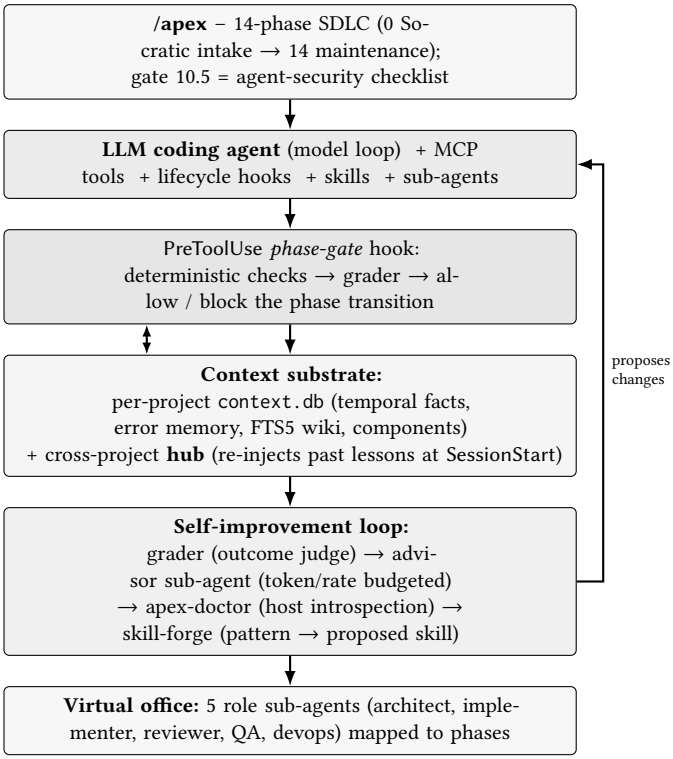


Figure 1: APEX layers on the agent's own extensibility primitives. A PreToolUse hook gates phase transitions; the context substrate is both per-project and cross-project; a self-improvement loop proposes changes back to the harness.

2 BACKGROUND: THE AGENT HARNESS

APEX is built on the extensibility surface that modern coding agents share; we prototype on Claude Code [2, 14], but the design uses only generic primitives. **Tool servers** (the Model Context Protocol [1]) let an agent call out to user-defined functions over a typed interface. **Lifecycle hooks** are scripts the agent invokes at fixed points—SessionStart, UserPromptSubmit, PreToolUse, PostToolUse—and a PreToolUse hook can *block* the call it intercepts. **Skills** are Markdown files with YAML front-matter (a name and a trigger description) that the agent loads on demand, turning a procedure into a first-class, model-selectable capability. **Sub-agents** are scoped child sessions with their own tool whitelist and model, dispatched for a bounded task. APEX uses all four: an MCP server for the context store and lifecycle operations, hooks for the gates and for context injection, skills for the development procedures, and sub-agents for the advisor and the per-phase "virtual office" roles. No model weights, no fork of the agent, no API keys are required to run the harness; the agent's own subscription is the only LLM.

3 APEX ARCHITECTURE

Figure 1 shows the layers. We describe each in turn.

3.1 A phase-gated software lifecycle

The /apex command runs a 14-phase lifecycle: phase 0 is a Socratic intake that decides whether the idea is well-formed enough to skip the interview; phases 1–3 cover problem framing, design (with a competitor scan and a validated DESIGN.md carrying real tokens), and a checkpointed spec; phase 4 enumerates edge cases *before* code; phases 5–7 are TDD implementation [19, 24]; phase 8 integration review; phase 9 QA with a coverage floor; phase 10 a security audit; **gate 10.5** maps an eight-layer agent-security checklist to the work and *blocks phase 11 unless every layer passes*; phases 11–14 are deploy, launch, scale, and maintenance. The security gate's value is less the checklist than the mandatory, dedicated attention it forces: a controlled study found that directing reviewers to focus on security multiplied vulnerability detection several-fold, while adding a bespoke checklist on top did not help further [5]; and where checklists do help complex, high-stakes work, the gain tracks enforced compliance rather than the list itself [4]—so APEX makes the security pass a blocking phase, not an optional list. The command file also encodes eleven concrete "do not repeat" errors drawn from past projects.

The discipline is enforced, not advisory. A PreToolUse hook intercepts the apex_set_phase tool. It first runs *deterministic* checks—is git initialised and are there commits? do the tests pass? is the OS what the stack assumed? are there emoji in user-facing strings? is the required schema present? do declared integration points resolve to real code rather than mocks?—and only if those pass does it consult the outcome-based grader (§3.3). If either says "block", the transition is refused with an actionable message. This "deterministic-first" ordering is deliberate: cheap, reproducible checks gate the common failures; the LLM judge is reserved for what genuinely needs judgement. That a formal gate before advancing reduces downstream defects is long established [8, 10]; more recent evidence shows gates work best when *selective*—risk-based gating of a minority of changes cut production outages by half at scale without throttling throughput [17]—which is why APEX gates only phase transitions, not every action.

3.2 The context substrate

Each project gets a `.apex/context.db` (SQLite, 25 tables, FTS5). It stores **temporal facts**—decisions and components carry `valid_from` / `invalid_from` timestamps, so superseded choices are retained but marked stale rather than deleted (a Graphiti-style bi-temporal model); **a bidirectional error memory**—each recorded error has a `recurrence_count` and links to the decision that should prevent it, so a repeat increments rather than duplicates; **a wiki**—Markdown pages with front-matter and wiki-links, full-text indexed; and **progressive disclosure**—the same context is retrievable at three sizes (compact ≈200 tokens, timeline ≈800, full ≈2000) so a session pays only for the depth it needs. Retrieval is hybrid (BM25 over FTS5 plus keyword overlap). Capturing decision rationale near the code is known to aid maintenance [7], yet manual adoption of architecture decision records stays low [6]; APEX addresses the cost by writing them automatically as part of the lifecycle.

A second database, the **hub**, aggregates across all APEX projects on the machine. Its job is anti-amnesia at the *ecosystem* level: a SessionStart hook reads the hub and injects, at the top of every new

session in any APEX project, the critical errors recorded *elsewhere*—so the SwiftUI-on-Linux mistake made in one project becomes a guardrail in the next.

3.3 The self-improvement loop

Four components let the harness critique and patch itself. **The grader** is an outcome-based judge: given a claimed accomplishment and the project state, it runs deterministic checks (git status, test results, OS assumptions, emoji, presence of Pydantic models / Playwright E2E, etc.) and, when enabled, an LLM judge; it emits a verdict with a `should_block` flag that the phase-gate hook honours. This ordering is a deliberate response to the known limits of LLM-as-judge: strong models agree with human raters on code tasks with a clear rubric [28, 30], but agreement collapses on hard correctness pairs [26]—so APEX gates on cheap deterministic checks first and reserves the judge for what genuinely needs judgement. **The advisor** is an Opus sub-agent invoked behind a strict budget—it sees a bundle capped at ≈2500 tokens, will not run unless at least eight tool calls have elapsed since its last review, and a circuit breaker disables it above three calls per minute—and returns short "advice cards" of four kinds (forgotten-tool, phase-drift, recommend-skill-handoff, grader-overdue) that a UserPromptSubmit hook surfaces to the operator. **apex-doctor** runs three analysers over the context store—clusters of errors with high recurrence or severity, grader-block patterns recurring across phases, and stale hand-off files—and writes idempotent, deduplicated proposals; it is triggered by *intent* (a wrap-up phrase, or a session older than a threshold), not by a cron. **The skill forge** watches the tool-call log and the grader history; when it sees the same blocking issue across ≥3 phases, a 3-tool sub-sequence repeated ≥5 times, or a decision recurring across ≥2 projects, it drafts a candidate SKILL.md into a proposals directory for the operator to approve or reject (rejections are remembered). It can also ingest community skills from public repositories, validating front-matter and scanning for leaked secrets before proposing. Finally, **reasoning traces**—short, categorised records of why the agent did what it did at key gates—are logged per phase and exportable in ShareGPT format, both as an audit trail and as future fine-tuning fuel.

3.4 The virtual office

Five role sub-agents—architect, implementer, reviewer, QA, devops—are installed as native sub-agents with their own tool whitelists and APEX-aligned system prompts (no emoji; Spanish prose for this user; consult prior decisions before proposing a stack; never ship mock data). A phase→role map recommends, e.g., the implementer for phases 5–7 and the reviewer for phase 8; the orchestrator dispatches the role via the agent's own Task mechanism. Because everything runs under the operator's subscription, the office costs no API spend.

4 USING APEX

Install. Clone `apex-harness`, register the MCP server and the hooks with the agent (a one-line config), and install the skills and role sub-agents. **Run.** In a project directory, invoke /apex. On a new idea the agent runs the Socratic round, then proposes an architecture and a checkpointed spec; phase 4 forces an edge-case list; phases 5–7 implement test-first, one vertical slice at a time, with git initialised at the very first sub-step; phase 8 reviews the integration; phase 9 enforces the coverage floor and runs Playwright E2E against the *real* backend; phase 10 plus gate 10.5 do the security pass and block deployment until clean; phases 11–14 ship and maintain it. Throughout, every decision, error, and component is written to `context.db` and mirrored to the hub; the advisor periodically nudges; apex_get_context re-grounds the agent at any point at the cheapest sufficient depth. On an *existing* codebase, a reverse-engineering path scans the repo and back-fills the substrate before the lifecycle resumes. The accompanying screencast shows a session end to end: /apex starting, the phase-gate refusing a premature transition, an advisor card firing, the grader verdict, and the hub injecting a cross-project lesson.

5 EVIDENCE OF UTILITY

APEX is not yet the subject of a controlled study; what we can report is the following. The artifact ships **620 passing tests** covering the context schema and retrieval, the grader and goal loop, the hand-off and token-budget machinery, the doctor's analysers, the skill forge's detectors, the role registry, and the prototype scaffolder. It has been the **working harness for ≈20 real software and research projects** over roughly two months—web applications, multi-agent systems, several university final-year projects and their tooling, research-workflow tools, and APEX's own development (it is built with itself). Across those projects the substrate has accumulated **322 recorded decisions, 184 recorded errors** (with recurrence tracking and links to preventive decisions), **279 components, and 160 wiki pages**, all aggregated in the cross-project hub; the SessionStart injector currently re-surfaces five `critical` cross-project lessons in every new session—among them, from one project, "propose a platform-dependent stack only after checking the OS", "never use emoji as UI icons", "do not mark an integration phase complete against mock data", and "initialise git as the first sub-step of implementation", and from another, "data detected by an upstream agent must flow explicitly through the shared context, not be re-derived downstream". These were real mistakes; they are now harness-enforced guardrails.

We are explicit about what is *not* validated. The self-improvement loop (grader, advisor, doctor, skill forge) is implemented and unit-tested but has only just begun to be exercised in the field: the production databases hold a single grader verdict, three doctor proposals, and one reasoning trace at the time of writing. We therefore claim a working, self-instrumenting harness with substantial field use of its memory layer—not measured improvement from its optimiser layer. Quantifying that is the obvious next step.

6 RELATED WORK

Harness engineering and externalization. Zhou et al. [31] survey the shift "from weights to context to harness" and identify *self-evolving harnesses* and *shared infrastructure* as open directions; APEX is a concrete instance of the former with the latter built in (the hub). "Dive into Claude Code" [14] documents the very substrate APEX builds on (MCP, plugins, skills, hooks, sub-agents, layered compaction) and lists open design questions; we answer one of them with a working layer. **Self-evolving agents.**

Fang et al.'s survey [9] frames the four-part loop in which APEX's grader/advisor/doctor/skill-forge are *Optimisers* acting on the *Agent System*. SEVerA [3] constrains self-evolving agents with first-order-logic output contracts and formal verification—a heavyweight alternative to APEX's pragmatic, SQL-predicate phase gates; the two are complementary (one could verify a phase gate's predicate). **Coordination as a layer.** Nechepurenko and Shuvalov [21] argue coordination should be a separate, independently measured architectural layer in multi-agent systems; APEX applies the same separation to a single agent—phase gating and role mapping are the coordination layer, distinct from the agent's capability. **Autonomous-agent frameworks.** Early autonomous loops (AutoGPT, BabyAGI [20, 25]) and orchestration libraries (LangGraph [12], AutoGen [29]) provide the loop and the plumbing but not a phase-gated SDLC, a temporal cross-project memory, or a self-tuning step; APEX is closer to a methodology made executable than to a library. Multi-agent development frameworks that decompose work into ordered software-engineering roles—MetaGPT [11] and ChatDev [22]—establish the phase-as-assembly-line principle APEX inherits, but verify artifacts by agent judgement rather than a deterministic gate.

Concurrent harnesses. We frame the design space by five capabilities: (a) phase gates over an explicit lifecycle, (b) deterministic pre-LLM blocking, (c) an outcome-driven self-improvement loop, (d) cross-project memory transfer, and (e) running on an unmodified commercial agent. The individual pillars of APEX are, as of mid-2026, each addressed by concurrent work, and we do not claim any one in isolation. *Deterministic pre-LLM gating* (b) is not exclusive: Rel(AI)Build [15] enforces a phase state machine through the same PreToolUse mechanism on an unmodified Claude Code—"enforced prior to LLM invocation"—and AgentSpec [27] generalises trigger/check/enforce interception before an action executes; OpenCastle [18] fail-closes on secret scans before any other gate. *Cross-project memory* on an unmodified agent is delivered head-to-head by ProjectMem [16], whose library-level lessons surface in a later project sharing the same dependency—though its gate is advisory, not blocking. *Outcome-driven self-improvement* appears in RHO [23], which optimises an agent harness from past trajectories, and in Meta-Harness [13], which optimises scaffolding around a fixed model. What none of these combine is the intersection: Rel(AI)Build has phase gating and blocking but, by its authors' own hedge, only a "detection mechanism, not a validated behavioural predictor" for adaptation, and no cross-project transfer; RHO self-improves but has neither lifecycle phase gates nor cross-project memory; ProjectMem carries lessons across projects but neither gates phases nor closes an outcome loop. To the best of our knowledge, no prior *or concurrent* system pairs an outcome-judged, fail-closed loop that tunes the harness itself (grader, advisor, doctor, skill-forge) *on top of* deterministic phase-transition gates *and* a cross-project memory hub—the cells $(a)\cap(b)\cap(c)$ and $(c)\cap(d)$ remain empty across the systems we surveyed. APEX's contribution is that composition on an unmodified commercial agent, not any single gate.

7 LIMITATIONS AND AVAILABILITY

Limitations. (1) Running the lifecycle end to end requires a commercial coding agent (we prototype on Claude Code) and its paid subscription; the artifact itself—the MCP server, hooks, skills, and the 620-test suite—is fully inspectable and the tests run standalone, but a reviewer cannot reproduce a full agentic session without that platform. (2) Evaluation is single-user so far; a multi-user study is needed and planned. (3) The substrate is local-first SQLite; team synchronisation beyond the file-based hub is future work. (4) Several skills encode this user's conventions (Spanish prose, anti-emoji); they are configuration, not core, but a reviewer should know. **Availability.** Source: https://github.com/vjrivmon/apex-harness (MIT). Archived snapshot with DOI: https://doi.org/10.5281/zenodo.XXXXXXX (to be minted). Screencast (≤5 min, voiced): https://youtu.be/XXXXXXXXXXX. Reproducing the test suite: `PYTHONPATH=lib .venv/bin/pytest tests/`.

ACKNOWLEDGMENTS

This work was supported by VRAIN, Universitat Politècnica de València.

REFERENCES

[1] Anthropic. 2024. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol. Accessed 2026-05.
[2] Anthropic. 2025. Claude Code. https://docs.anthropic.com/en/docs/claude-code. Accessed 2026-05.
[3] Debangshu Banerjee, Changming Xu, Eugene Ie, Ming Zhang, Daiyi Peng, Chu-Cheng Lin, and Gagandeep Singh. 2026. SEVerA: Verified Synthesis of Self-Evolving Agents. arXiv:2603.25111. arXiv:2603.25111 [cs.LG]
[4] Annegret Borchard, David L. B. Schwappach, Aline Barbir, and Paula Bezzola. 2012. A Systematic Review of the Effectiveness, Compliance, and Critical Factors for Implementation of Safety Checklists in Surgery. *Annals of Surgery* 256, 6 (2012), 925–933. https://doi.org/10.1097/sla.0b013e3182682f27
[5] Larissa Braz, Christian Aeberhard, Gül Çalıklı, and Alberto Bacchelli. 2022. Less is More: Supporting Developers in Vulnerability Detection during Code Review. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3510003.3511560
[6] Georg Buchgeher, Stefan Schoberberger, and Rainer Weinreich. 2023. Using Architecture Decision Records in Open Source Projects—An MSR Study on GitHub. *IEEE Access* 11 (2023). https://doi.org/10.1109/access.2023.3287654
[7] Sergio Cozzetti B. de Souza, Nicolas Anquetil, and Káthia M. de Oliveira. 2005. A Study of the Documentation Essential to Software Maintenance. *Proceedings of the 23rd Annual International Conference on Design of Communication (SIGDOC)* (2005), 68–75. https://doi.org/10.1145/1085313.1085331
[8] Michael E. Fagan. 1986. Advances in Software Inspections. *IEEE Transactions on Software Engineering* SE-12, 7 (1986), 744–751. https://doi.org/10.1109/tse.1986.6312976
[9] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. arXiv:2508.07407. arXiv:2508.07407 [cs.AI]
[10] Donald E. Harter, Mayuram S. Krishnan, and Sandra A. Slaughter. 2000. Effects of Process Maturity on Quality, Cycle Time, and Effort in Software Product Development. *Management Science* 46, 4 (2000), 451–466. https://doi.org/10.1287/mnsc.46.4.451.12056
[11] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352. arXiv:2308.00352 [cs.AI]
[12] LangChain. 2024. LangGraph. https://github.com/langchain-ai/langgraph. Accessed 2026-05.
[13] Yoonho Lee et al. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv:2603.28052. arXiv:2603.28052 [cs.LG]
[14] Jiacheng Liu, Xiaohan Zhao, Xinyi Shang, and Zhiqiang Shen. 2026. Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems. arXiv:2604.14228. arXiv:2604.14228 [cs.SE]
[15] Padmaraj Madatha. 2026. Rel(AI)Build: A Deterministic Control Plane for LLM Coding Agents. arXiv:2606.26924. arXiv:2606.26924 [cs.SE]
[16] Étienne Malo and Yizhou Qiu. 2026. ProjectMem: A Local-First, Event-Sourced Memory and Judgment Layer for Coding Agents. arXiv:2606.12329. arXiv:2606.12329 [cs.SE]

[17] Audris Mockus et al. 2025. Leveraging Risk Models to Enable Code Un-Freeze at Scale. *ACM Transactions on Software Engineering and Methodology* (2025). https://doi.org/10.1145/3722216

[18] Monki Labs. 2026. OpenCastle: A Validation-Gated, Self-Improving Multi-Agent Harness. https://github.com/monkilabs/opencastle. Accessed 2026-07.

[19] Nachiappan Nagappan, E. Michael Maximilien, Thirumalesh Bhat, and Laurie Williams. 2008. Realizing Quality Improvement through Test Driven Development: Results and Experiences of Four Industrial Teams. *Empirical Software Engineering* 13, 3 (2008), 289–302. https://doi.org/10.1007/s10664-008-9062-z

[20] Yohei Nakajima. 2023. BabyAGI. https://github.com/yoheinakajima/babyagi. Accessed 2026-05.

[21] Maksym Nechepurenko and Pavel Shuvalov. 2026. Coordination as an Architectural Layer for LLM-Based Multi-Agent Systems. arXiv:2605.03310. arXiv:2605.03310 [cs.MA]

[22] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924. arXiv:2307.07924 [cs.SE]

[23] RHO authors. 2026. RHO: Retrospective Harness Optimization for Self-Improving Coding Agents. arXiv:2606.05922. arXiv:2606.05922 [cs.SE] City University of Hong Kong and Microsoft Research Asia.

[24] Adrian Santos, Sira Vegas, Markku Oivo, and Natalia Juristo. 2020. A Family of Experiments on Test-Driven Development. *Empirical Software Engineering* 25, 5 (2020), 3595–3644. https://doi.org/10.1007/s10664-020-09895-8

[25] Significant Gravitas. 2023. AutoGPT. https://github.com/Significant-Gravitas/AutoGPT. Accessed 2026-05.

[26] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2024. JudgeBench: A Benchmark for Evaluating LLM-Based Judges. arXiv:2410.12784. arXiv:2410.12784 [cs.LG]

[27] Haoyu Wang, Christopher M. Qin, Zeyu Bai, Zhenting Wang, Mengnan Zhang, and Guanhong Chen. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666. arXiv:2503.18666 [cs.SE] ICSE 2026.

[28] Ruiqi Wang et al. 2025. Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *Proceedings of the ACM on Software Engineering* (2025). https://doi.org/10.1145/3728963

[29] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. arXiv:2308.08155 [cs.AI]

[30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2306.05685 [cs.CL]

[31] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. 2026. Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering. arXiv:2604.08224. arXiv:2604.08224 [cs.AI]